

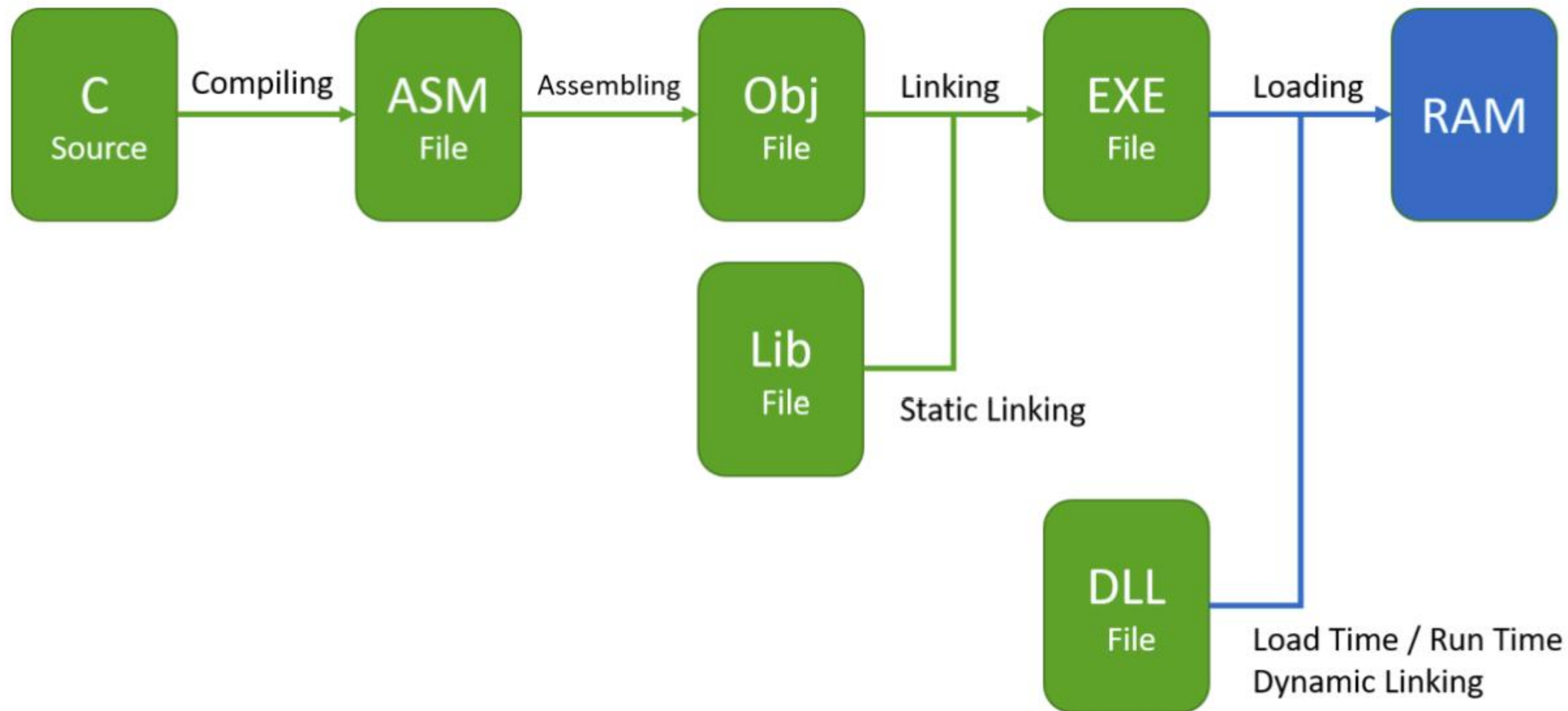
פורמט PE ותהליך
Loading

מערכות הפעלה

פורמט PE ותהליך LOADING

בפרק הקודם למדנו על תהליך היצירה של קבצי exe ו-DLL מתוך קבצי מקור בשפת C. עברנו על שלבי ה-Compiling, Assembling וה-Linking. הזכרנו באופן כללי שיש שלב נוסף, שלב ה-Loading, שבו הקבצים נטענים לזיכרון ה-RAM. עקב המורכבות של תהליך ה-Loading ראוי להקדיש לו פרק נפרד. האיור הבא ממחיש את השלב אותו נלמד בפרק זה, לעומת שלבים שכיסינו בפרק הקודם.

פורמט PE ותהליך LOADING



פורמט PE ותהליך LOADING

בפרק זה נעסוק בתפקידי ה-Loader, נלמד כיצד קבצים מאורגנים בפורמט שנקרא PE, קיצור של Portable Executable. זהו פורמט שמאגד את המידע שה-Loader צריך בשביל לבצע את משימותיו.

LOADER

LOADER

תפקידי ה-Loader

נניח ויצרנו קובץ exe. הקובץ כולל פקודות בשפת מכונה, כתובות בהן הפקודות נמצאות, משתנים, כתובות בהן המתשנים נמצאים, פונקציות והכתובות שלהן. האם המעבד יכול להריץ את הקובץ? חישבו על כך לפני שתמשיכו, מה חסר?

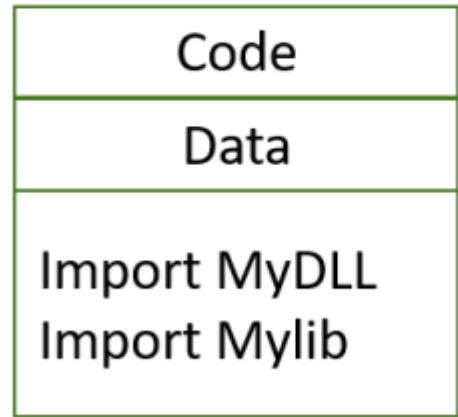
LOADER

תפקידי ה-Loader

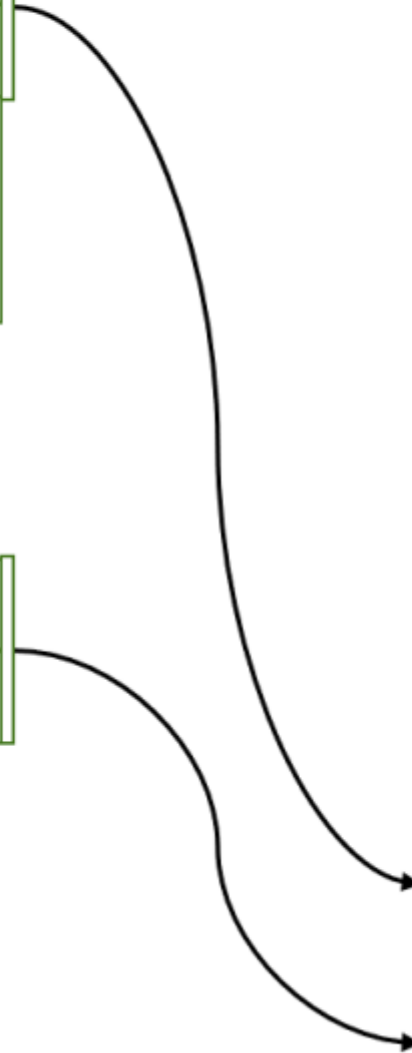
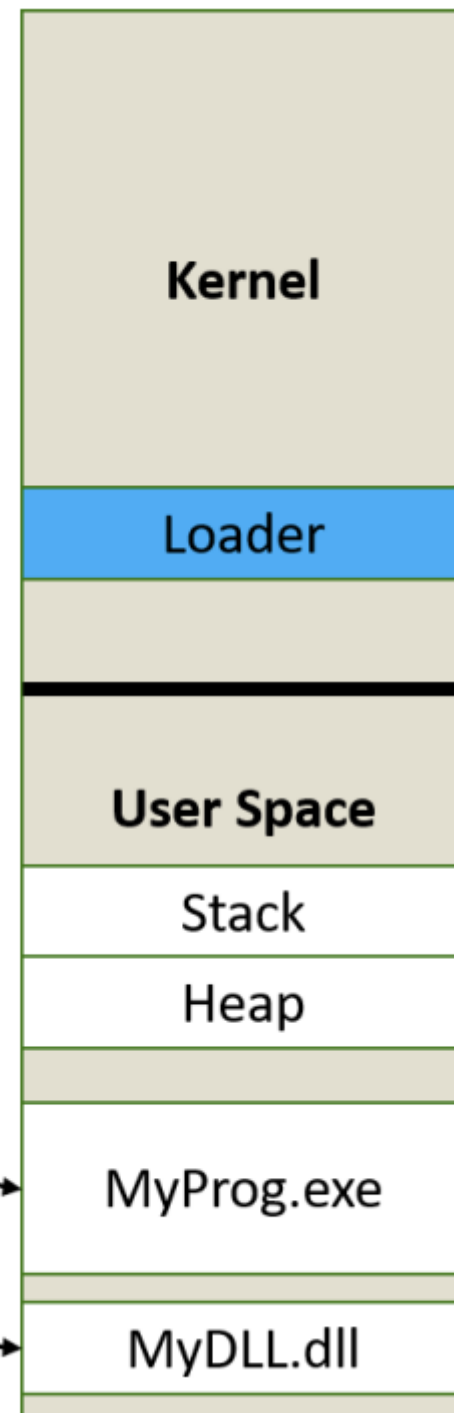
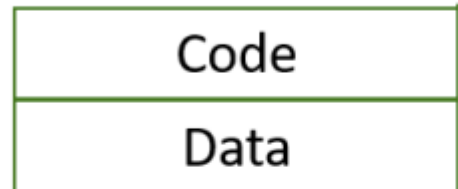
הדברים שחסרים הם:

1. טעינה לזיכרון. קובץ ה-exe שמור בהתקן זיכרון כגון דיסק קשיח, תקליטור או USB. המעבד לא יכול להריץ קבצים מהתקנים אלו, מכיוון שכפי שראינו אין לו יכולת לגשת לכתובת ספציפית בזיכרון ההתקנים הללו אלא רק לבלוק שמכיל כתובות רבות. לכן, תפקידו הראשון של ה-Loader הוא לטעון את הקובץ לזיכרון ה-RAM, ומכאן שמו- Loader, טוען. האיור הבא ממחיש את אופן פעולת ה-Loader. יש ברשותנו קוד שמבצע Static Linking לקובץ MyLib.lib ובנוסף מבצע Dynamic Linking לקובץ MyDLL.dll. הקוד שבוצע לו Static Linking הופך לחלק בלתי נפרד מקובץ ה-exe. את קובץ ה-DLL, לעומת זאת, ה-Loader יטען בנפרד.

MyProg.exe



MyDLL.dll



LOADER

תפקידי ה-Loader

הדברים שחסרים הם:

2. פתרון כתובות (Address Resolution). בזמן ביצוע ה-Linking, לתוך הקוד בשפת מכונה נכנסו כמעט כל הכתובות שצריך לגשת אליהם בזיכרון. אך יש משהו שה-Linker לא ידע: לאילו כתובות ייטענו פונקציות שאנחנו מייבאים מתוך DLLים. היזכרו בכך ש-DLLים נמצאים בזיכרון משותף ויכולים לשמש Process רבים. כאשר קובץ הרצה נטען לזיכרון, עליו להשתמש בכתובות של פונקציות שכבר נמצאות בזיכרון. ה-Linker לא יכל לחזות מה יהיה מיפוי הזיכרון בזמן שיוחלט להריץ את התוכנית, בין היתר עקב מנגנון רנדומלי בשם ASLR שיורחב עליו בהמשך. לכן ה-Linker משאיר את המלאכה הזו ל-Loader. מיד נראה, שלעיתים ה-Loader לא רק פותר כתובות שה-Linker לא ידע לפתור, אלא גם משנה כתובות שה-Linker פתר.

LOADER

תפקידי ה-Loader

לכן, אם נסכם עד כה, תפקידי ה-Loader הם: טעינה לזיכרון, פתרון כתובות, טעינה של DLLים שנטענים ב-Load Time (בניגוד ל-DLLים שנטענים ב-Run Time, כפי שלמדנו בפרק הקודם), הקצאת זיכרון למשתנים לא מאותחלים והפניית המעבד לנקודת תחילת הריצה.

LOADER

המחשה ל-Address Resolution על ידי ה-Loader

קטע הקוד הבא מבוסס על תכנית שכתבנו בפרק הקודם. הקוד קורא לפונקציה Share, אשר נמצאת ב-dll שיצרנו. נפתח את קובץ ה-exe באמצעות IDA. שימו לב לכתובת של הפונקציה Share כפי שה-Linker קבע:

```
.text:004116FE B9 03 B0 41 00      mov     ecx, offset unk_41B003
.text:00411703 E8 FB FA FF FF      call    sub_411203
.text:00411708 8B F4               mov     esi, esp
.text:0041170A FF 15 E0 A0 41 00    call    ds:Share
```

כלומר זוהי הכתובת 0x0041A0E0 (אם אינכם מבינים מדוע הכתובת "מבולבלת", היזכרו ב-Little Endian). כדי לצפות במרחב הכתובות הוירטואלי של פרוסס לאחר טעינתו ל-RAM אנחנו צריכים לעבוד עם תוכנה שמסוגלת להראות לנו את הזיכרון בזמן ריצה. נשתמש בתוכנה שטרם הכרנו, WinDbg. לא חייבים לדעת להשתמש בתוכנה בשביל להבין את ההסברים הבאים

usemydll!main:

009c16e0 55 push ebp

009c16e0 57 push edi
009c16ec 8dbd40ffffff lea edi, [ebp-0C0h]
009c16f2 b930000000 mov ecx, 30h
009c16f7 b8cccccccc mov eax, 0CCCCCCCCh
009c16fc f3ab rep stos dword ptr es:[edi]
009c16fe b903b09c00 mov ecx, offset usemydll!_AD1F52AD_usemydll.cpp (009cb003)
009c1703 e8fbfaffff call usemydll!ILT+510(__CheckForDebuggerJustMyCode (009c1203))
009c1708 8bf4 mov esi, esp
009c170a ff15e0a09c00 call dword ptr [usemydll!_imp__Share (009ca0e0)]
009c1710 3bf4 cmp esi, esp
009c1712 e8f6faffff call usemydll!ILT+520(__RTC_CheckEsp) (009c120d)
009c1717 33c0 xor eax, eax
009c1719 5f pop edi
009c171a 5e pop esi
009c171b 5b pop ebx
009c171c 81c4c0000000 add esp, 0C0h
009c1722 3bec cmp ebp, esp
009c1724 e8e4faffff call usemydll!ILT+520(__RTC_CheckEsp) (009c120d)
009c1729 8be5 mov esp, ebp
009c172b 5d pop ebp
009c172c c3 ret

להלן ה-main כפי שנמצא בזיכרון (שימו לב לכך שזו תוצאה של בחירת כתובת בהרצה ספציפית אחת, בהרצות אחרות הכתובות צפויות להשתנות):

LOADER

```
009c1708 8bf4      mov     esi, esp
009c170a ff15e0a09c00 call    dword ptr [usemydll!_imp__Share (009ca0e0)]
009c1710 3bf4      cmp     esi, esp
009c1712 e8f6faffff call    usemydll!ILT+520(  RTC CheckEsp) (009c120d)
```

איך נראית עכשיו הקריאה לפונקציה Share? יש קריאה מסוג dword ptr[009CA0E0]

בשפת אסמבלי המשמעות היא שזהו פשוט מצביע לזיכרון. צריך לגשת לכתובת 0x009CA0E0 ושם נמצא את הכתובת של הפונקציה המבוקשת, Share. אם כך נעשה זאת.

Memory

Address: 009ca0e0

00000000`009CA0E0	00	10	F5	77	00	00	00	00	00	00	00	00	00	00	00	00	00
00000000`009CA0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00000000`009CA100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

בכתובת זו נמצאת הכתובת 0x77F51000.

LOADER

בסופו של התהליך הגענו למסקנה ש-Share נמצאת בכתובת 0x77F51000. מסתבר שאפשר להגיע אל התשובה הזו בדרך פשוטה הרבה יותר, WinDbg מספק לנו את הכתובות של הפונקציות השונות:

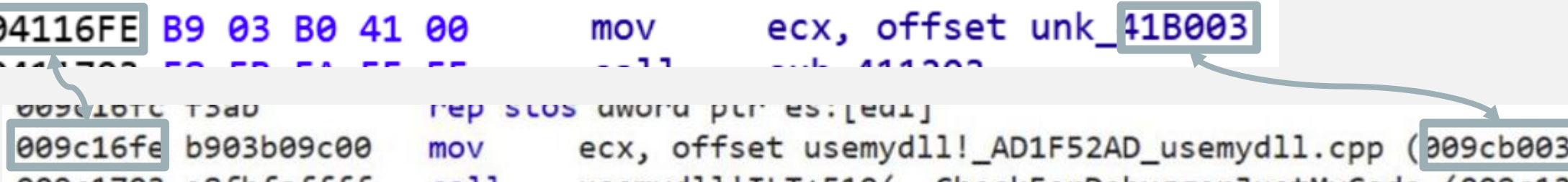
```
0:000> x mydll!Share
77f51000      mydll!Share (void)
```

ורואים שזו אכן הכתובת.

LOADER

אם תחזרו אל הקוד של פונקציית ה-main תשימו לב לכך שכל הכתובות זזו. הכתובת של המשתנה הגלובלי, שמועתק לתוך ecx לפני הקריאה לפונקציה, זזה מ- 0x0041B003 ל- 0x009CB003. אפילו הפקודות זזו: שורת הקוד של ההעתקה הנ"ל עברה מהכתובת 0x004116FE אל הכתובת 0x009C16FE.

```
.text:004116FE B9 03 B0 41 00 mov ecx, offset unk_41B003
00411700 50 50 5A 55 55 rep stos dword ptr es:[edi]
009C16FE b903b09c00 mov ecx, offset usemydll!_AD1F52AD_usemydll.cpp (009cb003)
```



בטח שמתם לב לכך שיש תבנית קבועה להזזות הכתובות. נראה שה-Loader החליף את כל הכתובות באותו אופן- במקום להתחיל ב-0x0041, הכתובות מתחילות ב-0x009C. יתר הבתים שמייצגים את הכתובת נותרו זהים.

LOADER

מה ה-Loader לא שינה? ובכן, יש שני דברים שלא השתנו. הראשון, הפקודות עצמן. כלומר היכן שהיתה פקודה להעתיק כתובת לרגיסטר ECX, לדוגמה, הפקודה נותרה העתקה ל-ECX. היכן שהיתה פקודה להעתיק את הרגיסטר ESP לתוך ESI, הפקודה נותרה ללא שינוי. הדבר השני שלא השתנה היא הפקודה השניה, שמבצעת Call לפונקציה כלשהי. שפת המכונה היתה E8FBFAFFFF ונותרה ללא שינוי. במחשבה ראשונה זה נראה מוזר, מכיוון שאם כל הכתובות זזו אז גם הפונקציה הזו היתה צריכה לעבור כתובת. אולם, הקריאה לפונקציה הזו מתבצעת באמצעות קפיצה *יחסית*. את נושא הקפיצות היחסיות סקרנו במהלך ההסבר על תהליך ה-Linking, כאשר סקרנו כיצד קוד בשפת C מומר לשפת מכונה. בקצרה, הפקודה אומרת למעבד לקרוא לפונקציה שנמצאת כמות מסויימת של בתים מסוף פקודה זו. כיוון שכל הקוד מוזז באותה כמות בתים, הפונקציה נותרה במרחק זהה של בתים מהשורה שקוראת לה, לכן ה-Loader לא היה צריך לבצע תיקון כתובת.

LOADER

ביצוע Relocation

נכיר מונח חדש- Relocation. במקרה שלנו, לא מדובר על לקחת את המשפחה ולנסוע לחו"ל מטעם העבודה, אלא בתהליך שינוי הכתובות בזיכרון. כל כתובת שה-Linker שם במקום מסויים וה-Loader העביר לכתובת אחרת, עברה Relocation. כפי שראינו, ה-Relocation מבוצע בהזזה קבועה לכל הכתובות. בהמשך הפרק נבין יותר לעומק את התהליך, אולם העקרון הוא פשוט למדי: ישנה כתובת בסיס, שנקבעת על ידי ה-Linker, ויש טבלה בה ה-Linker כותב את כל הכתובות שצריכות לעבור Relocation אם ה-Loader מחליט לא להשתמש בכתובת הבסיס של ה-Linker אלא לבצע הזזת כתובות.

LOADER

נסקור דוגמה שהתרחשה לנגד עינינו. כך נראתה פקודת ההעתקה של משתנה גלובלי כלשהו לתוך הרגיסטר ecx, כפי שקבע ה-Linker:

```
.text:004116FE B9 03 B0 41 00 mov ecx, offset unk_41B003
```

ה-Linker מכניס לטבלת ה-Relocation את הכתובת 0x004116FF (כן, בית אחד לאחר תחילת הפקודה, כיוון ששם מתחיל החלק של הכתובת, שרק אותו צריך לשנות). כל כניסה בטבלה נקראת "Fix Up", כיוון שהיא מצביעה על מקום שצריך "לתקן" בקוד.

ה-Loader החליט לבצע Relocation, במקרה שלנו להזיז אותן בהיסט של 0x005B0000 (זה ההפרש בין 0x00410000 לבין 0x009C0000). המשתנה שנמצא בכתובת 0x0041B003 זז יחד עם כל אזור הזיכרון של ה-data. הבעיה היא שה-Loader צריך גם לתקן את כל הפניות אל המשתנה שיש בקוד. הוא נכנס לטבלה ורואה את ה-Fix Up, שבכתובת 0x004116FF יש פניה למשתנה. כעת ה-Loader יודע איפה לבצע את התיקון.

LOADER

```
.text:004116FE B9 03 B0 41 00 mov ecx, offset unk_41B003
```

הנה, כך נראה הקוד לאחר התיקון, כפי שמצאנו אותו בזמן הריצה, לאחר שה-Loader שינה אותו:

```
009c16fe b9 03 b0 9c 00 mov ecx, offset usemydll!_AD1F52AD_usemydll.cpp (009cb003)
```

כפי שרואים, הכתובת של המשתנה עברה תיקון והיא שונתה בזמן הטעינה מהדיסק ל-RAM.

פורמט PE

פורמט PE

כדי שה-Loader יוכל לבצע את עבודתו, הוא צריך לקבל את כל המידע שנחוץ לו והמידע צריך להיות מאורגן באופן מסודר. צריך לדעת היכן נמצא הקוד, היכן הנתונים, היכן יש כתובות שצריך לבצע להן Relocation, היכן יש פונקציות שצריך לייבא, היכן נקודת ההתחלה של הריצה ועוד. במילים אחרות, צריך פורמט לקבצים מסוג קבצי הרצה וקבצי .DLL הפורמט הזה נקרא במערכת Windows פורמט PE, קיצור של Portable Executable. בפורמט PE נעשה שימוש גם בסוגים אחרים של קבצים שאינם מענייננו כעת, כגון קבצי SCR שומרי מסך, קבצי CPL, OCX, SYS ועוד. במערכת לינוקס יש פורמט אחר, הקרוי ELF, קיצור של Executable and Linkable Format.

לפני שנצלול פנימה לתוך פורמט PE, צריך להדגיש שזהו פורמט מורכב הכולל מגוון עצום של פרטים. אנחנו נתמקד בפרטים החשובים ביותר, ואף על פי כן הדיון צפוי להיות ארוך. מצד שני, הבנת הפורמט תאפשר לנו לבצע דברים מעניינים ולא שגרתיים.

נשתמש בתוכנה CFF Explorer

פורמט PE

אזור ה-DOS Header

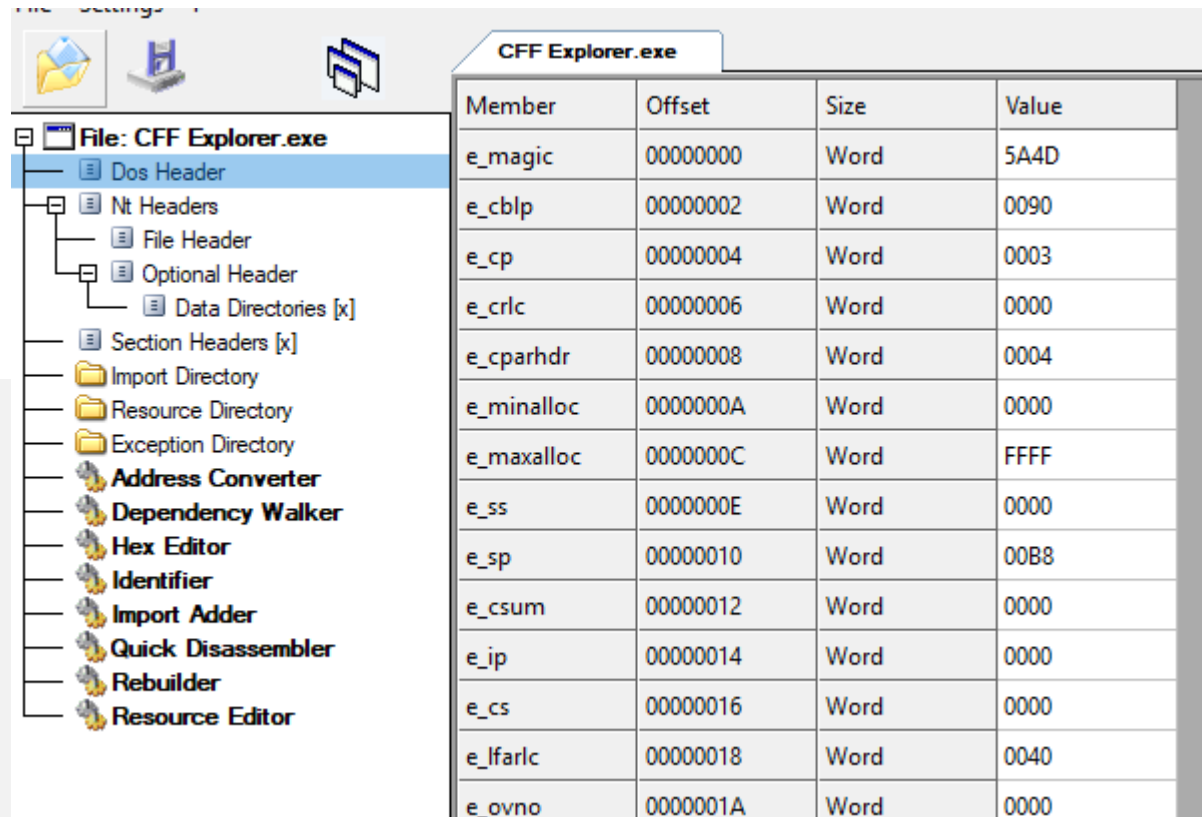
הקטע הראשון בקובץ PE נקרא DOS Header. הסיבה שקוראים לו כך תתברר מיד.

פיתחו קובץ הרצה exe כלשהו באמצעות CFF Explorer. אתם יכולים לראות שה-DOS Header כולל מגוון

שדות. בפועל, יש רק 2 שדות חשובים:

– שדה ה-Magic, השדה הראשון

– שדה ה-lfanew, השדה האחרון



The screenshot shows the CFF Explorer application window. On the left is a tree view with 'File: CFF Explorer.exe' expanded, showing 'Dos Header' selected. On the right is a table of DOS Header fields.

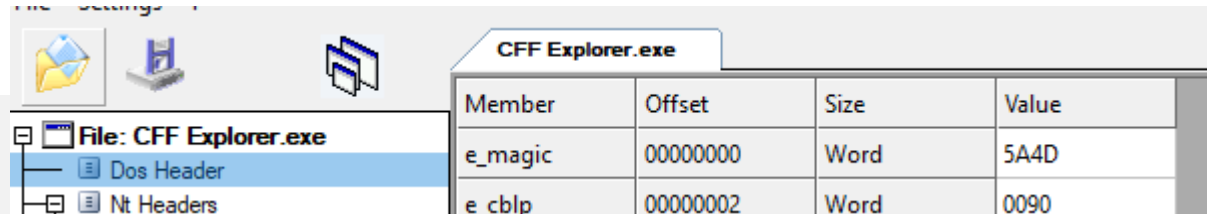
Member	Offset	Size	Value
e_magic	00000000	Word	5A4D
e_cblp	00000002	Word	0090
e_cp	00000004	Word	0003
e_crlc	00000006	Word	0000
e_cparhdr	00000008	Word	0004
e_minalloc	0000000A	Word	0000
e_maxalloc	0000000C	Word	FFFF
e_ss	0000000E	Word	0000
e_sp	00000010	Word	00B8
e_csum	00000012	Word	0000
e_ip	00000014	Word	0000
e_cs	00000016	Word	0000
e_lfanlc	00000018	Word	0040
e_ovno	0000001A	Word	0000

פורמט PE

שדה ה-Magic

שדה זה חייב להכיל את הקוד "5A4D", שמיתרגם בקוד ASCII לאותיות "MZ". זוהי מחווה למי שהמציא את

פורמט PE, ששמו Mark Zbikowski.

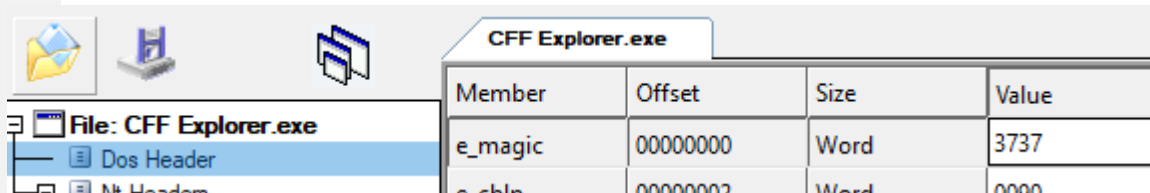


The screenshot shows the CFF Explorer application window. On the left, a tree view shows the file structure: File: CFF Explorer.exe, Dos Header, and Nt Headers. The main pane displays a table of PE header fields.

Member	Offset	Size	Value
e_magic	00000000	Word	5A4D
e_cblp	00000002	Word	0090

פיתחו קובץ exe או DLL כלשהו באמצעות CFF Explorer. גשו למסך Hex Editor ושנו את MZ למשהו

אחר. שימרו את הקובץ (בתפריט יש אופציה Save As).



The screenshot shows the CFF Explorer application window after modification. The 'e_magic' field value has been changed from 5A4D to 3737.

Member	Offset	Size	Value
e_magic	00000000	Word	3737
e_cblp	00000002	Word	0090

נסו להריץ את הקובץ

מה קרה?

This app can't run on your PC

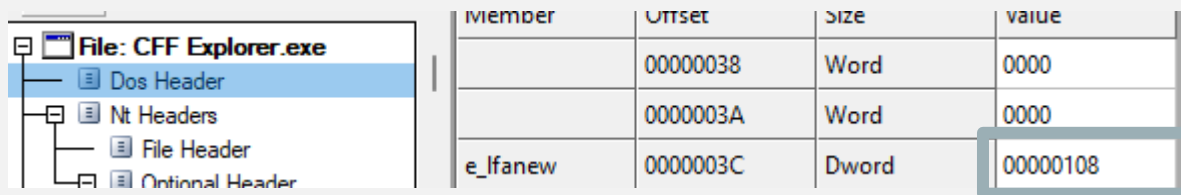
To find a version for your PC, check with the software publisher.

Close

פורמט PE

שדה ה-lfanew

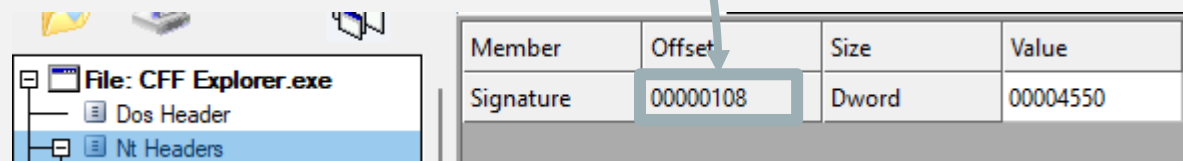
שדה זה כולל את המיקום של ה-Header הבא, ה-NT Header.



The screenshot shows the CFF Explorer interface. On the left, a tree view displays the file structure: File: CFF Explorer.exe, Dos Header, Nt Headers, File Header, and Optional Header. On the right, a table lists the members of the PE header.

Member	Offset	Size	Value
	00000038	Word	0000
	0000003A	Word	0000
e_lfanew	0000003C	Dword	00000108

באמצעות CFF Explorer, בידקו מהו הערך שנמצא ב-lfanew. הכנסו ל-NT Header ובידקו היכן הוא מתחיל (ההתחלה נמצאת בשדה שנקרא Signature), וודאו שהמיקום הוא אותו מיקום.



The screenshot shows the CFF Explorer interface with the 'Nt Headers' section selected in the tree view. On the right, a table lists the members of the NT Headers.

Member	Offset	Size	Value
Signature	00000108	Dword	00004550

פורמט PE

לאחר שביצעתם את התרגיל, ראוי להדגיש שהערך שמצאתם יכול להיות שונה, אפשר לערוך אותו ועדיין הפורמט יהיה תקין. זאת בניגוד לערך MZ, שאם נשנה אותו הקובץ לא יהיה שמיש. מדוע מישהו ירצה להזיז את מיקום ה-NT Header? ובכן, התשובה טמונה בדרך שבה פועלות תוכנות אנטי וירוס. תוכנות אלו סורקות קבצים במחשב ומחפשות קבצים זדוניים. כדי לעשות זאת הן מנתחות קבצי הרצה, אך בלי להריץ אותם כמובן. לכותבי נוזקות יש ערך רב במציאת דרכים ליצור קבצי הרצה שמצד אחד יהיו תקינים לפי הגדרות הפורמט, אך מצד שני יכשילו את האנטי וירוסים באמצעות סידורים מורכבים ולא סטנדרטיים של קובצי ההרצה.

פורמט PE

אזור ה-NT Header

ה-NT Header מכיל 3 דברים:

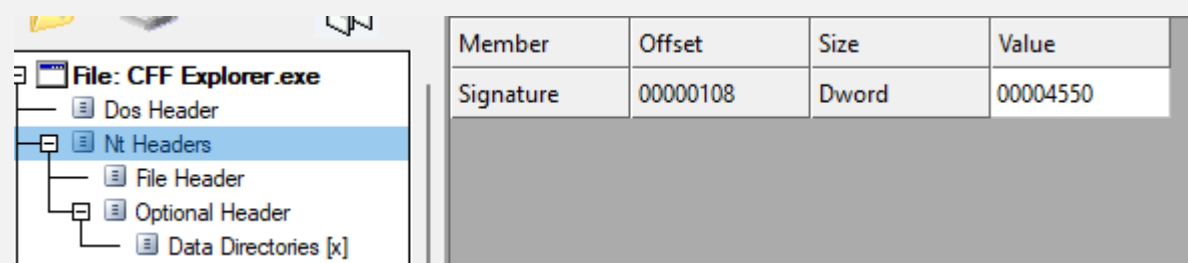
1. שדה בשם Signature. שדה זה חייב להיות 45 50, שהוא קוד ה-ASCII של "PE".

2. מבנה נתונים בשם File Header

3. מבנה נתונים בשם Optional Header

מעבר לכך אין מה להרחיב על ה-NT Header, המידע המעניין נמצא בכל אחד ממבני הנתונים שהוא כולל

בתוכו



The screenshot shows the CFF Explorer application. On the left, a tree view displays the file structure: File: CFF Explorer.exe, Dos Header, Nt Headers (selected), File Header, Optional Header, and Data Directories [x]. On the right, a table displays the Signature field:

Member	Offset	Size	Value
Signature	00000108	Dword	00004550

פורמט PE

Member	Offset	Size	Value	Meaning
Machine	00000104	Word	014C	Intel 386
NumberOfSections	00000106	Word	0005	
TimeDateStamp	00000108	Dword	5D443FFC	
PointerToSymbolTable	0000010C	Dword	00000000	
NumberOfSymbols	00000110	Dword	00000000	
SizeOfOptionalHeader	00000114	Word	00E0	
Characteristics	00000116	Word	0102	Click here

אזור ה-File Header

ה-File Header הוא מבנה נתונים שנראה כך:

ה-Offset שכתוב ליד שם של כל שדה, הוא המקום בקובץ ה-exe שבו נמצא השדה הנ"ל. לדוגמה, שדה ה-TimeDateStamp נמצא 0x108 בתים מתחילת קובץ ה-exe. נדגיש, כי באופן כללי שדה ה-Offset מתייחס למיקום על הדיסק הקשיח, לא הכתובת בזיכרון ה-RAM שאליו נטען הקובץ. אפשר לראות שבין ה-Offsetים יש הפרשים של מספר בתים. לדוגמה שדה ה-Machine נמצא 0x104 בתים מתחילת הקובץ, גודלו כפי שנראה מיד הוא 2 בתים, ולכן השדה הבא יתחיל ב-Offset של 0x106. מבין השדות ב-File Header נעבור על אלו שמובלטים בכחול.

פורמט PE

Member	Offset	Size	Value	Meaning
Machine	00000104	Word	014C	Intel 386
NumberOfSections	00000106	Word	0005	
TimeDateStamp	00000108	Dword	5D443FFC	

Machine

שדה זה מציין על איזה סוג ארכיטקטורה של מעבד אמור לרוץ הקובץ. השדה יכול להכיל אחד משני ערכים: הערך 014C מציין מעבד 32 ביט. בהמשך נראה שזה אינו השדה היחיד שבו מוגדר סוג המעבד.

NumberOfSections

שדה זה, כשמו כן הוא. מציין את מספר ה-Sections בקוד.

TimeDateStamp

שדה זה מציין את הזמן בו נוצר הקובץ על ידי הקומפיילר, לפי שיטת הספירה של UNIX. בשיטה זו, סופרים שניות החל מהשעה 00:00:00 בתאריך ה-1 לינואר 1970, לפי שעון UTC (זמן לונדון, ללא התחשבות בשעון קיץ).

פורמט PE

Member	Offset	Size	Value	Meaning
Machine	00000104	Word	014C	Intel 386
NumberOfSections	00000106	Word	0005	
TimeDateStamp	00000108	Dword	5D443FFC	
PointerToSymbolTable	0000010C	Dword	00000000	
NumberOfSymbols	00000110	Dword	00000000	
SizeOfOptionalHeader	00000114	Word	00E0	
Characteristics	00000116	Word	0102	Click here

Characteristics

הקליקו על "Click Here" בשורה של Characteristics. תקבלו טבלה כזו:

☒ File is executable
☐ File is a DLL
☐ System File
☐ Relocation info stripped from file
☐ Line numbers stripped from file
☐ Local symbols stripped from file
☐ Agressively trim working set
☐ App can handle >2gb address space
☐ Bytes of machine word are reversed (low)
☒ 32 bit word machine
☐ Debugging info stripped from file in .DBG file
☐ If Image is on removable media, copy and run from the swap
☐ If Image is on Net, copy and run from the swap file
☐ File should only be run on a UP machine
☐ Bytes of machine word are reversed (high)

OK Cancel

פורמט PE

השדות שנעסוק בהם הם:

- File is executable - ללא הסימון לא ניתן להריץ את הקובץ
- File is a DLL - יש בסך הכל ביט אחד שמבדיל בין קובץ exe ו-DLL!
- Relocation info stripped from file - כזכור, ה-Loader עשוי לתקן כתובות בזיכרון, אם הוא אינו משתמש בכתובות שה-Linker יצר. לטובת התיקון, יש צורך בטבלה שאומרת אילו כתובות צריך לתקן. אם שדה זה מסומן, הטבלה הזו לא קיימת בקובץ ולכן ה-Loader לא יוכל לבצע תיקון כתובות. המשמעות המעשית היא שה-Loader יטען לזיכרון בדיוק באותן כתובות שה-Linker קבע.
- 32 bit word machine - שוב, שדה שקובע אם הקובץ הוא 32 ביט או 64.

פורמט PE

אזור ה-Optional Header

השם Optional הוא שם מטעה ביותר, מכיוון שהוא ממש לא אופציונלי. השם נקבע להיות כך בזמן שפורמט הקבצים תוכנן להיות מותאם לסוגים שונים של מערכות הפעלה, ולכן הנתונים של Windows היו אופציונליים עבור מערכות הפעלה אחרות. אבל עבור Windows, המידע שנמצא שם הוא הכרחי להרצה.

להלן השדות של ה-Optional Header של 32 ביט, אותם שדות קיימים גם ב-64 ביט אך הגדלים של חלקם שונים כמובן. מסומנים השדות בהם נעסוק.

Member	Offset	Size	Value	Meaning	Member	Offset	Size	Value	Meaning
Magic	00000118	Word	010B	PE32	MinorImageVersion	00000146	Word	0000	
MajorLinkerVersion	0000011A	Byte	0E		MajorSubsystemVersion	00000148	Word	0006	
MinorLinkerVersion	0000011B	Byte	0F		MinorSubsystemVersion	0000014A	Word	0000	
SizeOfCode	0000011C	Dword	00000E00		Win32VersionValue	0000014C	Dword	00000000	
SizeOfInitializedData	00000120	Dword	00001400		SizeOfImage	00000150	Dword	00006000	
SizeOfUninitializedData	00000124	Dword	00000000		SizeOfHeaders	00000154	Dword	00000400	
AddressOfEntryPoint	00000128	Dword	00001338	.text	Checksum	00000158	Dword	00000000	
BaseOfCode	0000012C	Dword	00001000		Subsystem	0000015C	Word	0003	Windows Console
BaseOfData	00000130	Dword	00002000		DllCharacteristics	0000015E	Word	8140	Click here
ImageBase	00000134	Dword	00400000		SizeOfStackReserve	00000160	Dword	00100000	
SectionAlignment	00000138	Dword	00001000		SizeOfStackCommit	00000164	Dword	00001000	
FileAlignment	0000013C	Dword	00000200		SizeOfHeapReserve	00000168	Dword	00100000	
MajorOperatingSystemVersion	00000140	Word	0006		SizeOfHeapCommit	0000016C	Dword	00001000	
MinorOperatingSystemVersion	00000142	Word	0000		LoaderFlags	00000170	Dword	00000000	
MajorImageVersion	00000144	Word	0000		NumberOfRvaAndSizes	00000174	Dword	00000010	

הערה: השדה האחרון של ה-Optional Header הוא Data Directories, אשר ב-CFF Explorer מופיע בתור טאב נפרד. נתייחס אליו בקרוב, לאחר שנעבור על כל השדות המשמעותיים.

פורמט PE

Magic	00000118	Word	010B	PE32
-------	----------	------	------	------

Magic

קבצי הרצה שמיועדים למעבדים של 32 דורשים טיפול שונה מה-Loader לעומת קבצי הרצה שמיועדים למעבדי 64 ביט. לדוגמה, הכתובת שבה המעבד צריך להתחיל את הריצה: בקובץ שמיועד ל-32 ביט הכתובת תהיה שמורה בגודל של 32 ביט ואילו בקובץ של 64 ביט צריך כמובן לבטא את הכתובת כ-64 ביט. לכן, ה-Optional Header של 64 ביט צריך להיות שונה בכמה מקומות מאשר בגרסה של 32 ביט.

כדי שה-Loader יידע איך לקרוא את הנתונים, האם בשדות בגודל 32 או 64 ביט, הוא צריך לדעת אם ה-Optional Header הוא 32 או 64 ביט. השדה שקובע זאת הוא שדה ה-Magic. עבור 32 ביט הערך יהיה 10B, כמו בתמונה.

פורמט PE

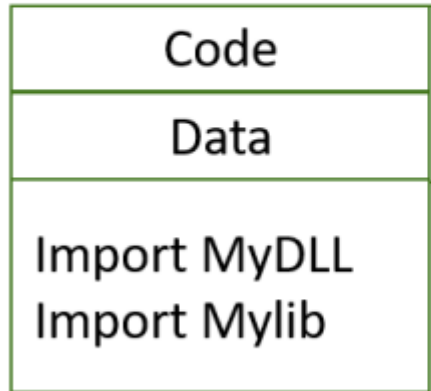
ImageBase	00000134	Dword	00400000	
-----------	----------	-------	----------	--

Image Base

אנחנו נדלג לרגע על שדה ה-Address of Entry Point ונתחיל בהסבר על Image Base, מסיבות שתיכף יתבררו.

כזכור, בזמן הרצת הקובץ, ה-Loader יעתיק את המידע מקובץ ההרצה אל מרחב הכתובות הוירטואלי של ה-Process שיווצר. במרחב הכתובות הזה נמצא הקוד של מערכת ההפעלה, של DLLים שנטענים, של זיכרון Stack ו-Heap שמוקצה, וגם כמובן לקובץ ה-exe יוקצה אזור זיכרון, החל בכתובת וירטואלית מסוימת. שדה ה-Image Base אומר מה ה-Linker מצפה שתהיה הכתובת הוירטואלית הזו.

MyProg.exe



MyDLL.dll

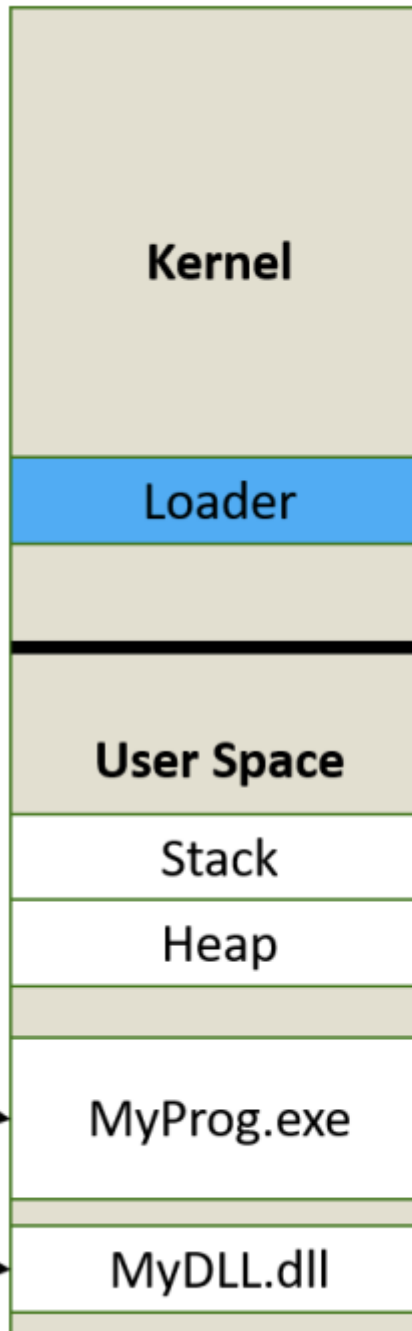
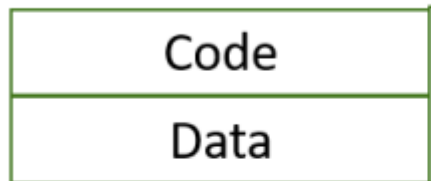
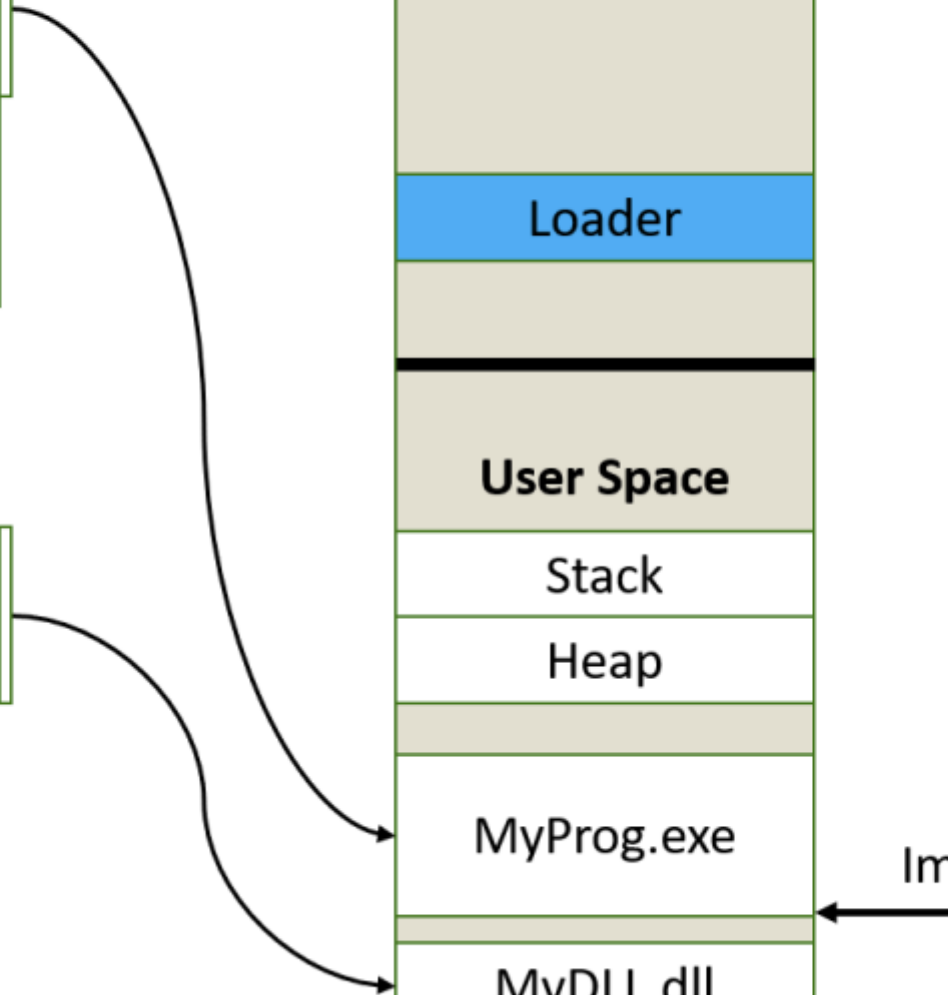


Image Base



פורמט PE

חישבו: למה המידע הזה חשוב? הרי ה-Loader יכול לטעון את הקובץ לזיכרון בכל כתובת וירטואלית, לא רק היכן שה-Linker מצפה לה?

התשובה היא שכפי שראינו ה-Linker מסדר את הכתובות בקובץ ה-exe בהתאם למיקום שבו הוא מבקש שה-Loader יטען אותם לזיכרון. אם ה-Loader רוצה לשנות את כתובת הטעינה לזיכרון, הוא צריך לתקן את הכתובות הללו, ולשם כך הוא צריך לדעת מה הניח ה-Linker. זהו תהליך ה-Relocations שסקרנו לפני כן. שדה ה-Image Base מאפשר ל-Loader להבין בכמה הוא צריך "להזיז" כתובות, כיוון שבלי לדעת מה כתובת הבסיס שקבע ה-Linker לא ניתן לדעת מה גודל ההזזה שצריך לבצע.

לדוגמה: ה-Linker קבע למשתנה מסויים כתובת של 0x00403000. ה-Linker הכניס ל-Image Base ערך של 0x00400000. כעת ה-Loader מבצע טעינה לזיכרון לא בכתובת שה-Linker ביקש, אלא לדוגמה בכתובת 0x00500000. לאיזו כתובת ה-Loader צריך לטעון את המשתנה?

ה-Loader יבצע חיסור של הכתובות, ויגלה כי המשתנה נמצא 0x3000 בתים מתחילת ה-Image Base. את ההיסט הזה הוא יוסיף לכתובת אליה הוא באמת טוען את הקובץ, ולכן המשתנה יהיה בכתובת וירטואלית 0x00503000.

פורמט PE

Address of Entry Point

שדה משמעותי זה קובע, כפי ששמו אומר, באיזו כתובת במרחב הכתובות הוירטואלי של ה-Process תתחיל ריצת המעבד ברגע שה-Process יוצר. נסביר איך להבין את הערך שרשום בשדה זה. להלן דוגמה:

Member	Offset	Size	Value	Meaning
AddressOfEntryPoint	00000128	Dword	00001338	.text

את שדה ה-Offset הכרנו כבר, הוא מתייחס למקום מתחילת הקובץ שבו נמצא הערך שאנחנו מחפשים. הערך עצמו הוא- במקרה זה – 0x1338. זהו ערך יחסי מה-Image Base אותו סקרנו זה עתה.

פורמט PE

AddressOfEntryPoint	00000128	Dword	00001338	.text
BaseOfCode	0000012C	Dword	00001000	
BaseOfData	00000130	Dword	00002000	
ImageBase	00000134	Dword	00400000	

לדוגמה, נניח כי ה-Image Base הוא 0x00400000 וכי ה-Loader לא מבצע שינוי אלא טוען לכתובת זו. במקרה זה, אם ערך ה-Address Of Entry Point הוא כפי שמופיע בדוגמה, אז ריצת הקוד תחל מכתובת 0x00401338 בזיכרון הוירטואלי.

נסיים בהסבר של מושג חדש: **RVA**. אלו ראשי תיבות של Relative Virtual Address. הכוונה לכתובת יחסית מתחילת ה-Image Base. במילים אחרות, בדוגמה שראינו 0x1338 הוא ה-RVA של מיקום תחילת ריצת הקוד. מעתה נשתמש במושג זה.

פורמט PE

SectionAlignment	00000138	Dword	00001000	
FileAlignment	0000013C	Dword	00000200	

Section Alignment, File Alignment

כאשר דברנו על מיפוי קבצים לזיכרון נתקלנו כבר בכך שהמיפוי אינו מתבצע סתם כך לכל כתובת בזיכרון, אלא לכתובות "עגולות". ראינו גם, שהזיכרון הוירטואלי מחולק ל-Pages, שגודלם בדרך כלל 4096 בתים (0x1000). גם המידע שאגור בהתקני אחסון כמו דיסק קשיח לא מפוזר סתם כך בכל כתובת, אלא מחולק לאזורי זיכרון שנקראים Sectors, ושגודלם הוא בדרך כלל 512 בתים (0x200).

המונח הטכני המדויק לכתובת "עגולה" הוא "Aligned". כאשר זיכרון הוא "Aligned" למספר מסוים, הכוונה היא שכתובת ההתחלה שלו היא כפולה של המספר. לדוגמה, זיכרון שהוא Aligned לגודל של Page, 0x10000, חייב להתחיל בכתובת שמסתיימת ב-000, כלומר 12 ביטים של אפסים.

פורמט PE

SectionAlignment	00000138	Dword	00001000	
FileAlignment	0000013C	Dword	00000200	

לכן אין זו הפתעה שגם בתהליך שבו קובץ ה- exe נטען לזיכרון, יש כתובות Aligned גם על הדיסק הקשיח וגם ב-RAM. שני הסוגים הללו של Alignment מוגדרים בקובץ ה-PE:

- שדה ה-File Alignment שמגדיר את ה-Alignment עבור אזורי הזיכרון על הדיסק הקשיח. בדרך כלל שדה זה יהיה 0x200, בדומה לגודל ההיסטורי של Sector בדיסק (כיום גודל Sector צפוי להיות שונה, לעיתים כ-4KB, אך פורמט PE עושה לעיתים שימוש בגודל ההיסטורי).
- שדה ה-Section Alignment, שמגדיר את ה-Alignment ב-RAM. בדרך כלל שדה זה יהיה 0x1000, בדומה לגודל הנפוץ של Page בזיכרון הוירטואלי.

פורמט PE

עיברו לטאב של Section Headers ותוכלו לצפות בארגון אזורי הזיכרון הן על הקובץ בדיסק והן מה שצפוי להיות כאשר הם ייטענו ל-RAM

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.data	0000000B	00001000	00000200	00000200
.text	00000140	00002000	00000200	00000400
.idata	000000D0	00003000	00000200	00000600

פורמט PE

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.data	0000000B	00001000	00000200	00000200
.text	00000140	00002000	00000200	00000400
.idata	000000D0	00003000	00000200	00000600

כדי לא להקדים את המאוחר, נתמקד כרגע רק בשתי עמודות:

- עמודת ה-Raw Address היא הכתובת בה מתחיל ה-Section בקובץ, על הדיסק. כפי שרואים, כל הכתובות מתחלקות יפה ב-0x200.
- עמודת ה-Virtual Address כוללת את ה-RVA (אנחנו משתמשים במונח החדש שלמדנו) של כל Section ב-RAM. כפי שרואים, כולם מתחלקים יפה ב-0x1000.

פורמט PE

כמובן, כאשר אנחנו מתכנתים אף אחד לא מצפה שנכתוב קוד שגודלו בדיוק כפולה של 0x200 או 0x1000. אם כך, מה קורה בקובץ ה-PE? נמצא את התשובה לבד.

הקליקו על ה-Text Section, בתחתית מסך ה-CFF Explorer תוכלו לראות את הבתים בקובץ על הדיסק, כפי שהם מאורגנים בכתובות יחסיות לתחילת ה-Section. גללו עד לסוף ה-Section, ותוכלו לראות מה שמור ברווח שבין סוף הקוד לסוף ה-Section:

פורמט PE

SizeOfImage	00000150	Dword	00006000	
-------------	----------	-------	----------	--

Size Of Image

שדה זה קובע ל-Loader כמה מקום יש להקצות ב-RAM עבור התוכנית כולה. המקום שיש להקצות הוא סכום המקום שצריך להיות מוקצה עבור כל אחד מה-Sections.

נבדוק זאת בעצמנו.

פיתחו את טאב ה-Data Sections ובידקו כמה זיכרון צריך להיות מוקצה עבור התוכנית

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.data	0000000B	00001000	00000200	00000200
.text	00000140	00002000	00000200	00000400
.idata	000000D0	00003000	00000200	00000600

פורמט PE

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.data	0000000B	00001000	00000200	00000200
.text	00000140	00002000	00000200	00000400
.idata	000000D0	00003000	00000200	00000600

ה-Section האחרון, idata, הוא כפי שראינו בעבר אזור הזיכרון שכולל את כל הקוד שמייבאים ממקורות אחרים, כגון DLLים. אזור הזיכרון מתחיל בכתובת 0x3000 וגודלו הוא 0x1000 (נזכור שגם אם הגודל קטן יותר, מתבצע ריפוד באפסים). לכן יש להקצות 0x4000 בתים עבור התוכנית. ואכן, זה ערכו של שדה Size Of Image:

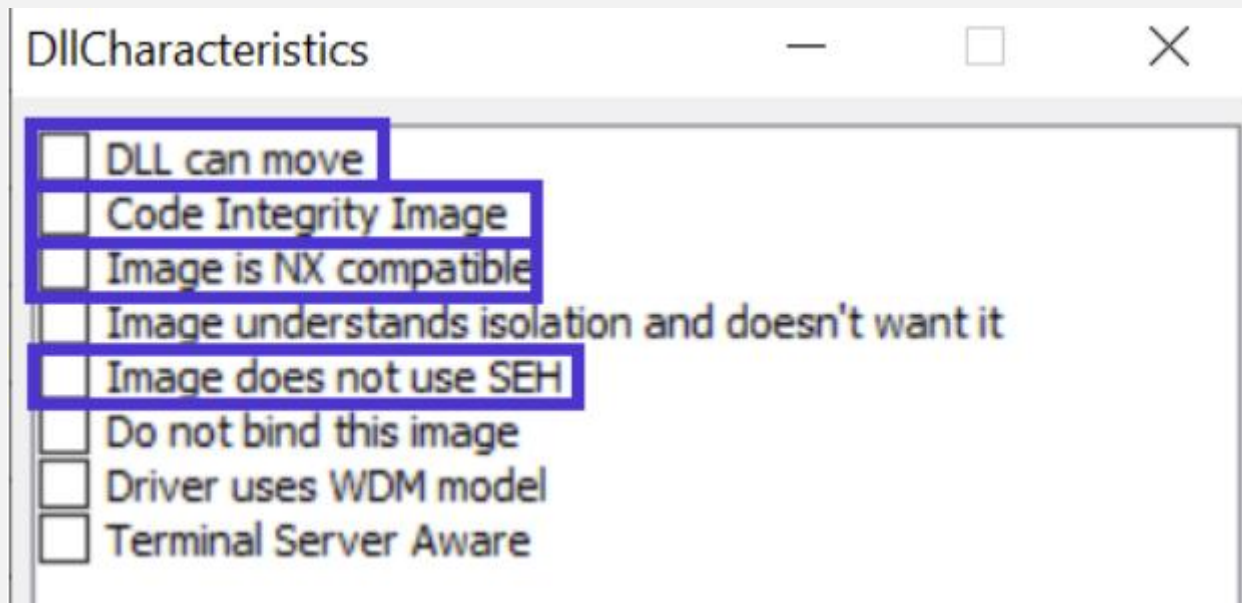
SizeOfImage	000000D0	Dword	00004000
-------------	----------	-------	----------

פורמט PE

DLL Characteristics

למרות שמו, המאפיינים שנמצאים בשדה זה אינם קשורים רק ל-DLLים אלא גם לקבצי .exe. באופן כללי, מדובר באוסף של מאפיינים הקשורים לאבטחה.

להלן אוסף השדות מתוך CFF Explorer. שימו לב לכך ששמות השדות אינם זהים לשמות שמוגדרים באפיון של קבצי PE, לכן לכל שדה מובא בסוגריים השם ה"אמיתי" שלו. מודגשים השדות שנלמד עליהם:



פורמט PE

- ☐ DLL can move
- ☐ Code Integrity Image
- ☐ Image is NX compatible
- ☐ Image understands isolation and doesn't want it
- ☐ Image does not use SEH

– שדה DLL can move (שם מדויק: Dynamic Base). משמעות: בזמן ה-Loading, אפשר לבצע שינויי כתובות. למעשה, מדובר במנגנון ASLR, עליו נפרט בהסבר נפרד.

– שדה Code integrity image (שם מדויק: Force Integrity). שדה זה מאלץ לבדוק אם החתימה הדיגיטלית של הקובץ תקפה. נסביר בקיצור רב, הסבר מלא הוא מעבר למיקוד של ספר זה: כפי שראינו, אפשר לערוך קבצי PE באופן ידני. יכולת זו מאפשרת לכותבי תוכנות זדוניות לשתול קוד בתוך תוכנות "טובות" ובעקבות זאת נוצר צורך במנגנון כלשהו שיבדוק אם קובץ PE הוא מקורי או שהקובץ עבר שינוי שיכול לגרום לו להיות זדוני. מנגנון האימות הנ"ל נקרא "חתימה דיגיטלית". בשדה שאנחנו דנים בו מוגדר האם צריך לבדוק שהחתימה הדיגיטלית תקפה לפני הרצה. כמובן שגם את השדה הזה ניתן לשנות ידנית (ולהפוך את הבדיקה ללא הכרחית), אבל החתימה הדיגיטלית מבוססת בין היתר על הערך בשדה הזה ולכן שינוי של השדה יגרום לתוכנות אבטחה להתערב לפני הרצת הקובץ.

פורמט PE

- ☐ DLL can move
- ☐ Code Integrity Image
- ☐ Image is NX compatible
- ☐ Image understands isolation and doesn't want it
- ☐ Image does not use SEH

– Image is NX compatible (שם מדויק: NX Compat). במעבדים יש דגל שנקרא NX (קיצור של No-Execute), כאשר הדגל דולק לא תתאפשר הרצת פקודות מאזורי זיכרון שאינם קוד. כלומר לא יהיה אפשרי להריץ קוד מהמחסנית, מ-data וכו'. כאשר השדה הזה מופעל בקובץ ה-PE, מערכת ההפעלה מדליקה ביט ב-PTE, שמסמן למעבד שאסור לו להריץ קוד מאזור זיכרון ללא הרשאות מתאימות.

– Image does not use SEH (שם מדויק: NO SEH). ה-SEH הוא קיצור של Structured Exception Handling. בקצרה, כאשר הקוד ביצע משהו חריג הריצה שלו תיעצר. זאת בניגוד למצב שבו הקוד קורא לקוד שתפקידו לטפל בשגיאה, מה שעלול לאפשר ניצול לרעה על ידי נוזקות.

פורמט PE

אזור ה-Data Directories

מבנה הנתונים שנקרא Data Directories הוא מערך של מצביעים. אפשר להתייחס אליו כאל מפה, שכתוב בה איפה אפשר למצוא נתונים שימושיים. דוגמאות שימושיות במיוחד הן:

- Imports: פונקציות שהקוד שלנו מייבא
- Exports: פונקציות שהקוד שלנו מייצא (היזכרו בכתיבת DLL, ניתן היה להגדיר אילו פונקציות יהיו זמינות למי שמייבא את ה-DLL)
- Resources: לדוגמה אייקונים שהתוכנה עושה בהם שימוש
- Relocations: רשימת כל הכתובות בקוד שצריך לתקן אם הקוד לא נטען ל-Base Address שה-Linker הניח

פורמט PE

מבנה הנתונים כולל 16 איברים, כל איבר נמצא במקום מוגדר מראש. לדוגמה, המידע על Imports תמיד יהיה באינדקס 0 בטבלה והמידע על Relocations תמיד יהיה באינדקס 5 בטבלה. כמובן שאם הנתון אינו קיים, לדוגמה אין טבלת Relocations, אז חלק מהנתונים הם ריקים (אפסים).

חישבו: במקרה שאין טבלת Relocations, מה יהיה הערך "DLL can move" ב-DLL Characteristics?
תשובה: כמובן, שאם אין מידע על Relocations, לא ניתן לבצע שינויי כתובות ולכן ה-DLL (או ה-exe) חייב להטען ב- Base Address שלו.

Member	Offset	Size	Value	Section
Export Directory RVA	00000190	Dword	00000000	
Export Directory Size	00000194	Dword	00000000	
Import Directory RVA	00000198	Dword	00288248	.rdata
Import Directory Size	0000019C	Dword	0000012C	
Resource Directory RVA	000001A0	Dword	002BF000	.rsrc
Resource Directory Size	000001A4	Dword	000A7D7C	
Exception Directory RVA	000001A8	Dword	002A6000	.pdata
Exception Directory Size	000001AC	Dword	00018120	

עבור כל נתון, רשומים הפרטים הבאים:

– ה-RVA שלו

– הגודל שלו

פורמט PE

אזור ה-Section Headers

מבנה נתונים זה מרכז את כל המידע לגבי ה-Sections השונים. אין שום מצביע שאומר מה המקום שלו בקובץ ה-PE, אבל קל להגיע אליו: הוא נמצא מיד אחרי ה-NT Header. המיקום של ה-NT Header נמצא בתוך ה-DOS Header, ואילו הגודל של ה-NT Header הוא קבוע, לכן למעשה מיקום ה-Section Headers נמצא ב-

$\text{DOS_Header} \rightarrow \text{ifanew} + \text{sizeof(NT Header)}$

להלן דוגמה, של ה-Sections הראשונים של Chrome.exe:

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00117184	00001000	00117200	00000400	00000000	00000000	0000	0000	60000020
.rdata	00031B1C	00119000	00031C00	00117600	00000000	00000000	0000	0000	40000040
.data	0000B118	0014B000	00003800	00149200	00000000	00000000	0000	0000	C0000040

פורמט PE

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00117184	00001000	00117200	00000400	00000000	00000000	0000	0000	60000020
.rdata	00031B1C	00119000	00031C00	00117600	00000000	00000000	0000	0000	40000040
.data	0000B118	0014B000	00003800	00149200	00000000	00000000	0000	0000	C0000040

נעבור על השדות המסומנים:

– Virtual Size: הגודל של Section-ה תופס בזיכרון הוירטואלי. המספרים כאן אינם מעוגלים ל-Section Alignment, אך בסופו של דבר המקום שנתפס בזיכרון הוא כמובן כפולה של ה-Alignment.

– Virtual Address: ה-RVA של Section-ה, כלומר מה המרחק של Section-ה מכתובת הבסיס שאליו ה-PE נטען. ה-RVA תמיד יהיה כפולה של ה-Section Alignment.

פורמט PE

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00117184	00001000	00117200	00000400	00000000	00000000	0000	0000	60000020
.rdata	00031B1C	00119000	00031C00	00117600	00000000	00000000	0000	0000	40000040
.data	0000B118	0014B000	00003800	00149200	00000000	00000000	0000	0000	C0000040

נעבור על השדות המסומנים:

– Raw Size: הגודל שה- Section תופס על הדיסק, כשהוא כבר מעוגל ל-File Alignment

– Raw Address: באיזו כתובת בקובץ ה-PE (כלומר על הדיסק) מתחיל ה-Section.

– Characteristics: אוסף של דגלים שמפרטים את ההגדרות של ה- Section. לדוגמה אם הוא כולל קוד, אם יש לו הרשאות קריאה, כתיבה, הרצה.

פורמט PE

אזור ה-Import Directory

כפי שראינו עד כה, הכתובות שאליהן נטענים DLLים לא יכולות להיות ידועות בזמן תהליך ה-Linking, ואפילו אם הן היו ידועות, עקב מנגנון ה-ASLR הן עשויות להשתנות בכל פעם שמערכת ההפעלה עולה עם הדלקת המחשב. לכן בכל מקרה שבו תוכנה עושה שימוש בפונקציות מתוך DLLים, ה-Loader יצטרך למצוא את הכתובות של הפונקציות ולתקן את הקוד בזמן הטעינה לזיכרון, כך שהפניות יהיו לכתובות הנכונות. ה-Import Directory כולל את כל המידע שה-Loader צריך כדי למצוא את הכתובות של הפונקציות הנדרשות.

כפי שמיד ניווכח, המידע לגבי ה-Import מאורגן בצורה מורכבת למדי, אבל בסופו של דבר העקרון הוא פשוט: טבלה שרשומים בה כל ה-DLLים שנעשה בהם שימוש, ובכל DLL יש רשימה של כל הפונקציות. בזמן ה-Loading, לצד כל פונקציה נוספת הכתובת שלה בזיכרון. הטבלה הנ"ל נקראת Import Address Table, או בקיצור IAT, והיא משמשת בזמן הריצה לקבוע בדיוק לאיזו כתובת צריך לבצע call כאשר קוראים לפונקציה מתוך DLL.

פורמט PE

למרות המורכבות של הפרטים, ההשקעה משתלמת. הבנה של ה-IAT תאפשר לנו לבצע תהליך שנקרא IAT Hooking, שבאמצעותו נשנה את אופן הביצוע של תוכנית תוך כדי ריצה.

נתחיל במשימה. השלב הראשון יהיה למצוא את ה-Import Directory בתוך קובץ ה-PE. שימו לב לכך ש-CFF Explorer נוטה להציג את השדות השונים במיקום שלהם ב-RVA, ולכן אם אנחנו רוצים לראות את מבנה הנתונים בקובץ ה-PE אנחנו צריכים לבצע מעט המרות כתובות. טענו את הקובץ Chrome.exe ובצעו את השלבים הבאים יחד עם ההסבר.

פורמט PE

1. מוצאו את ה-RVA של ה-Import Directory

לפי ה-Data Directory, ה-RVA של ה-Import Directory הוא 0x13F7FD0

Member	Offset	Size	Value	Section
Import Directory RVA	00000108	Dword	0013F7FD	.rdata

פורמט PE

2. מיצאו את הכתובת של ה-Import Directory בתוך קובץ ה-PE

כפי שכתוב, ה-Import Directory נמצא בתוך ה-rdata section. נפנה אל ה-Section Headers:

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.rdata	00031B1C	00119000	00031C00	00117600

אפשר לראות שתחילת ה-section הוא 0x117600 בתים מתחילת קובץ ה-PE (מה שנקרא "Raw Address") ושה-Section נטען ל-RAM החל מכתובת וירטואלית 0x119000 (נזכור שזהו מיקום יחסי למיקום שבו ה-Loader בחר לטעון את התוכנה, לדוגמה אם התוכנה נטענה לכתובת 0x00400000, אז ה-rdata Section נטען למקום 0x00519000 בזיכרון הוירטואלי).

פורמט PE

Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.rdata	00031B1C	00119000	00031C00	00117600

אנחנו רוצים להמיר את ה-RVA שערכו $0x0013F7FD$ לכתובת Raw, בקובץ. הטבלה שלפנינו מראה לנו שכתובת ה-RVA $0x00119000$ היא כתובת $0x00117600$ בקובץ. אם נחסר אותן נמצא כי ההפרש ביניהן הוא $0x1A00$. כלומר, כדי להמיר RVA כלשהו שנמצא בתוך rdata לכתובת בקובץ, צריך להחסיר ממנו $0x1A00$. נבצע זאת:

$$0x13F7FD - 0x1A00 = 0x13DDFD$$

מעולה, סיימנו שלב נוסף. כעת אנחנו יודעים היכן לחפש את ה- Import Directory בתוך הקובץ.

פורמט PE

3. מוצאו את ה-Import Directory בקובץ

ה-CFF Explorer מכיל בתוכו Hex Editor. ניגש אליו, ונקליק על החץ שכתוב עליו "Go To Offset". נזין את הכתובת שחישבנו. האזור המודגש ברקע שחור הוא ה-Import Directory

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0013DDF0	62	6F	78	65	64	50	72	6F	63	65	73	73	00	50	F8	13
0013DE00	00	00	00	00	00	00	00	00	00	10	17	14	00	30	FF	13
0013DE10	00	70	F8	13	00	00	00	00	00	00	00	00	00	1F	17	14
0013DE20	00	50	FF	13	00	10	FF	13	00	00	00	00	00	00	00	00
0013DE30	00	2C	17	14	00	F0	05	14	00	00	00	00	00	00	00	00
0013DE40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

כעת נפענח את המידע שיש בתוכו.

פורמט PE

כל רשומה ב-Import Directory מורכבת מ-5 שדות בגודל DWORD כל אחד. מיד נסקור אותם. הרשומות כתובות אחת אחרי השניה, בלי הפרדה. הדרך לדעת שהגענו לסוף ה-Import Directory היא רק כאשר הגענו לרשומה שמתחילה ב-DWORD שכולו אפסים. האיור הבא ממחיש שב-Import Directory של Chrome.exe יש 3 רשומות, ובכל רשומה 5 שדות:

[illegible]

פורמט PE

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0013DDF0	62	6F	78	65	64	50	72	6F	63	65	73	73	00	50	F8	13
0013DE00	00	00	00	00	00	00	00	00	00	10	17	14	00	30	FF	13
0013DE10	00	70	F8	13	00	00	00	00	00	00	00	00	00	1F	17	14
0013DE20	00	50	FF	13	00	10	FF	13	00	00	00	00	00	00	00	00
0013DE30	00	2C	17	14	00	F0	05	14	00	00	00	00	00	00	00	00
0013DE40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

נפענח את הרשומה הראשונה. הרשומה כוללת את הערכים הבאים:

1. 0x0013F850
2. 0x00000000
3. 0x00000000
4. 0x00141710
5. 0x0013FF30

כיוון שההסבר כולל פרטים רבים, אנחנו נתמקד בשדות 1,4 ו-5.

פורמט PE

שלושת השדות שנתמקד בהם כוללים כתובות RVA. כדי להבין על מה הם מצביעים, נרצה למצוא את הכתובות שלהם בקובץ. לכן, נחסיר מכל אחד מהם את ההיסט המתאים לקובץ הספציפי שלנו, 0x1A00.

1. $0x0013F850 - 0x1A00 = 0x0013DE50$
2. $0x00000000$
3. $0x00000000$
4. $0x00141710 - 0x1A00 = 0x0013FD10$
5. $0x0013FF30 - 0x1A00 = 0x0013E530$

נתחיל דווקא מהשדה הרביעי. שדה זה נקרא "Name" וכפי שאנחנו רואים הוא מצביע על הכתובת 0x0013FD10 בתוך הקובץ, כתובת שכאמור תהפוך להיות RVA 0x00141710 ברגע שהתוכנית תיטען ל-RAM. חפשו את הכתובת 0x0013FD10 ב-Hex Editor:

[illegible]

פורמט PE

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
0013FD10	63	68	72	6F	6D	65	5F	65	6C	66	2E	64	6C	6C	00	4B	chrome_elf.dll.K
0013FD20	45	52	4E	45	4C	33	32	2E	64	6C	6C	00	56	45	52	53	ERNEL32.dll.VERS
0013FD30	49	4F	4E	2E	64	6C	6C	00	00	00	00	00	00	00	00	00	ION.dll.....

כפי שרואים, זהו מצביע לשם של ה-DLL. אנחנו רואים שלושה שמות של DLLים, שמופרדים ביניהם ב-00.
כעת ברור למה שדה זה נקרא "Name" - הוא מצביע על השם של ה-DLL שנעשה בו שימוש.

פורמט PE

אך שם ה-DLL אינו מספיק, אנחנו צריכים לדעת גם אילו פונקציות בדיוק צריך לייבא מתוך ה-DLL. לכן נחקור את השדה הראשון. שדה זה נקרא "Original First Thunk", שם לא מהמוצלחים ביותר. למעשה, באמצעות השדה הזה פשוט מגיעים לרשימה של השמות של הפונקציות הספציפיות שצריך לייבא מכל DLL. נתחיל בתהליך החיפוש.

1. $0x0013F850 - 0x1A00 = 0x0013DE50$

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0013DE50	10	06	14	00	00	00	00	00	2C	06	14	00	00	00	00	00
0013DE60	3E	06	14	00	00	00	00	00	00	00	00	00	00	00	00	00

כפי שאפשר לראות, הכתובת 0x0013DE50 כוללת רשימה של 3 כתובות, שאחריהן יש רק אפסים (אחרי האפסים תתחיל רשימה של שמות הפונקציות של ה-DLL הבא). נמיר את הכתובות ל-Raw:

- $0x00140610 - 0x1A00 = 0x0013EC10$
- $0x0014062C - 0x1A00 = 0x0013EC2C$
- $0x0014063E - 0x1A00 = 0x0013EC3E$

פורמט PE

- $0x00140610 - 0x1A00 = 0x0013EC10$
- $0x0014062C - 0x1A00 = 0x0013EC2C$
- $0x0014063E - 0x1A00 = 0x0013EC3E$

נחפש את הכתובת הללו בקובץ ה-PE, ונמצא שם את שמות הפונקציות:

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
0013EC10	00	00	47	65	74	49	6E	73	74	61	6C	6C	44	65	74	61	..GetInstallData
0013EC20	69	6C	73	50	61	79	6C	6F	61	64	00	00	00	00	53	69	ilsPayload...Si
0013EC30	67	6E	61	6C	43	68	72	6F	6D	65	45	6C	66	00	00	00	gnalChromeElf...
0013EC40	53	69	67	6E	61	6C	49	6E	69	74	69	61	6C	69	7A	65	SignalInitialize
0013EC50	43	72	61	73	68	52	65	70	6F	72	74	69	6E	67	00	00	CrashReporting..

פורמט PE

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
0013EC10	00	00	47	65	74	49	6E	73	74	61	6C	6C	44	65	74	61	..GetInstallData
0013EC20	69	6C	73	50	61	79	6C	6F	61	64	00	00	00	00	53	69	ilsPayload...Si
0013EC30	67	6E	61	6C	43	68	72	6F	6D	65	45	6C	66	00	00	00	gnalChromeElf...
0013EC40	53	69	67	6E	61	6C	49	6E	69	74	69	61	6C	69	7A	65	SignalInitialize
0013EC50	43	72	61	73	68	52	65	70	6F	72	74	69	6E	67	00	00	CrashReporting..

שימו לב לכך שהכתובות מצביעות לא בדיוק על שמות הפונקציות, אלא שני בתים לפני שמות הפונקציות, שני בתים שבמקרה זה שווים 0x0000. בלי להכנס ליותר מדי פרטים, ניתן לייבא פונקציה לא לפי השם שלה אלא לפי המספר הסידורי שלה ב-DLL. שני בתים אלו הם שדה שנקרא "Hint".

פורמט PE

עד כאן הגענו למסקנה, שהנתונים ב-Import Directory הם:

- שמות ה-DLLים שמהם צריך לייבא פונקציות
- עבור כל DLL, השמות של הפונקציות שצריך לייבא מתוכו

מה נותר לנו לחפש? הדבר היחידי שחסר לנו, הוא הכתובות של הפונקציות הללו. ברגע שיהיו לנו גם את הכתובות של כל הפונקציות המיובאות, כל קריאה שלנו לפונקציה מתוך DLL כלשהו תתורגם בקלות לכתובת אליה יש לקפוץ.

פורמט PE

$$5. \quad 0x0013FF30 - 0x1A00 = 0x0013E530$$

נעבור לשדה החמישי והאחרון. שדה זה נקרא IAT, קיצור של Import Address Table, וכפי שאפשר להבין מהשם של טבלה זו, כאן שמורות הכתובות של הפונקציות המיובאות מה-DLLים. כפי שחישבנו לפני כן, הטבלה נמצאת בכתובת 0x0013E530 בקובץ ה-PE. גשו לכתובת זו.

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
0013E530	10	06	14	00	00	00	00	00	2C	06	14	00	00	00	00	00	0000000000000000
0013E540	3E	06	14	00	00	00	00	00	00	00	00	00	00	00	00	00	>0000000000000000

$$1. \quad 0x0013F850 - 0x1A00 = 0x0013DE50$$

מה קורה פה? אלו בדיוק הכתובות שנמצאות בשדה הראשון, ה-Original First Thunk. במקום הכתובות של הפונקציות בזיכרון, קיבלנו שוב מצביעים לשמות הפונקציות. חישבו על כך- מדוע קובץ ה-PE לא כולל את הכתובות של הפונקציות?

התשובה היא כמובן, שקובץ ה-PE לא יכול להכיל את הכתובות של הפונקציות מכיוון שהן לא ידועות בזמן ביצוע ה-Linking. אי אפשר לדעת לאן ה-DLLים ייטענו. את הכתובות הללו אפשר לבדוק רק בזמן ביצוע ה-Loading. אם כך, בואו נעשה זאת.

פורמט PE

כדי לבדוק את תמונת הזיכרון של תוכנה שעברה תהליך Loading, נשתמש גם הפעם ב-WinDbg.
באמצעות הפקודה lm (קיצור של List Modules), נבדוק להיכן נטען הקובץ chrome.exe

```
0:000> lm
```

start	end	<u>module name</u>
00007ff7`caf40000	00007ff7`cb0ec000	<u>chrome</u>
00007ff8`1ddd0000	00007ff8`1deb2000	<u>chrome_elf</u>
00007ff8`4a170000	00007ff8`4a17a000	<u>VERSION</u>
00007ff8`4ec30000	00007ff8`4ecbf000	<u>apphelp</u>
00007ff8`513f0000	00007ff8`51693000	<u>KERNELBASE</u>
00007ff8`52f40000	00007ff8`52fde000	<u>msvcrt</u>
00007ff8`537c0000	00007ff8`53872000	<u>KERNEL32</u>
00007ff8`54460000	00007ff8`54650000	<u>ntdll</u>

פורמט PE

```
0:000> lm
```

start	end	<u>module name</u>
00007ff7`caf40000	00007ff7`cb0ec000	<u>chrome</u>
00007ff8`1ddd0000	00007ff8`1deb2000	<u>chrome_elf</u>
00007ff8`4a170000	00007ff8`4a17a000	<u>VERSION</u>
00007ff8`4ec30000	00007ff8`4ecbf000	<u>apphelp</u>
00007ff8`513f0000	00007ff8`51693000	<u>KERNELBASE</u>
00007ff8`52f40000	00007ff8`52fde000	<u>msvcrt</u>
00007ff8`537c0000	00007ff8`53872000	<u>KERNEL32</u>
00007ff8`54460000	00007ff8`54650000	<u>ntdll</u>

שימו לב לכך שכל הכתובות הינן כתובות של 64 ביט (הקוד הורץ על מעבד ומערכת הפעלה של 64 ביט).

המודול chrome נטען לכתובת 0x00007FF7CAF40000. נתבונן בזיכרון בכתובת זו:

פורמט PE

00007ff7`caf40000 00007ff7`cb0ec000 [chrome](#)

המודול chrome נטען לכתובת 0x00007FF7CAF40000. נתבונן בזיכרון בכתובת זו:

Memory

Address:

00007FF7`CAF40000	4D	5A	78	00	01	00	00	00	04	00	00	00	00	00	00	00	00
00007FF7`CAF40010	00	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00

ואנחנו אכן רואים את "MZ", מספר הקסם 4D5A, שמציין את תחילת הקובץ שנטען ל-RAM.

פורמט PE

נעבור ל-IAT, שנמצא ב-RVA של 0x0013FF30 מתחילת הקובץ בזיכרון.

$$0x7FF7CAF40000 + 0x0013FF30 = 0x7FF7CB07FF30$$

Memory

Address: 00007ff7cb07FF30

00007FF7`CB07FF20	FE	16	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00
00007FF7`CB07FF30	74	3B	DD	1D	F8	7F	00	00	05	10	DD	1D	F8	7F	00	00	00
00007FF7`CB07FF40	00	10	DD	1D	F8	7F	00	00	00	00	00	00	00	00	00	00	00

כפי שאפשר לראות, הכתובות שנמצאות ב-IAT הוחלפו בתהליך ה-Loading. נותר לנו רק לוודא שהכתובות אכן מצביעות אל הפונקציות המתאימות בתוך ה-DLLים. הפונקציה הראשונה צריכה להיות GetInstallDetailsPayload מתוך chrome_elf.dll (אם לא ברור לכם מדוע, היזכרו בסדר של ה-DLLים ובסדר של הפונקציות בתוך ה-DLLים, כפי שראינו בתוך ה-Import Table).

פורמט PE

```
00007FF7`CB07FF30 74 3B DD 1D F8 7F 00 00 05 10 DD 1D F8 7F 00 00
00007FF7`CB07FF40 00 10 DD 1D F8 7F 00 00 00 00 00 00 00 00 00 00
```

ניקח את הכתובת הראשונה, כתובת של 64 ביט:

0x00007FF81DDD3B74

נשווה את הכתובת הזו לכתובת שבה נמצא GetInstallDetailsPayload מתוך chrome_elf.dll.

```
0:000> x chrome_elf!GetI*
00007ff8`1ddd3b74 chrome\_elf!GetInstallDetailsPayload (void)
```

וכפי שאפשר לראות, הכתובת שקיימת ב-IAT היא אכן הכתובת של הפונקציה שנמצאת ב-Import Table.

ASLR

ASLR

ראשי התיבות של ASLR הן Address Space Layout Randomization. כדי להבין מה פירוש הדברים, נתחיל מהבסיס. כפי שראינו, בקובץ ה-PE יש שדה של Base Address, שקובע את הכתובת המועדפת שאליה ייטען קובץ ה-exe או ה-DLL. זו כתובת מועדפת בלבד, מערכת ההפעלה לא חייבת להתחשב בהעדפה. למעשה, הגיוני שקבצי DLL לא ייטענו לכתובת המועדפת, מכיוון שסביר שהיא כבר תפוסה. עבור קבצי exe הכתובת המועדפת ככל הנראה לא תפוסה, משום שעם יצירת Process חדש הם הדבר הראשון שנטען לזיכרון, כשמרחב הזיכרון הוירטואלי עוד פנוי.

בעבר, כאשר מנגנון ASLR לא היה קיים, מערכת ההפעלה Windows היתה מבצעת Loading לכתובות ברירת מחדל: קבצי exe שלא ביקשו כתובת מועדפת היו נטענים לכתובת 0x00400000, ואילו קבצי DLL היו נטענים לכתובת 0x10000000. אם הכתובות הללו היו תפוסות, ה-Loader היה מבצע Relocation לכתובות אחרות.

ASLR

הדבר הקל על כותבי נוזקות, שיכלו להניח שחלקי קוד, אזורי זיכרון או DLLים יימצאו בכתובות מסויימות בזיכרון.

מנגנון ה-ASLR משפיע על ההיסט שבו ייטענו לזיכרון קבצי exe ו-DLLים ועל ההיסטים של ה-Stack וה-Heap עבור כל מידע שנטען למרחב הזיכרון של Process. באופן זה, לדוגמה, מי שמצליח לכתוב קוד ל-Stack ומעוניין לקפוץ ממנה חזרה אל ה-Code Section, לא יכול לדעת מראש את המרחק שיהיה עליו לקפוץ.

בניגוד ל-Stack ול-Heap, קבצי DLL דורשים שיתוף זיכרון ולכן אי אפשר לטעון אותם בכתובת שונה עבור כל Process. במקרה של DLLים, בזמן תהליך עליית מערכת ההפעלה נוצר על ידי מנהל הזיכרון מספר אקראי, אליו נטען ה-DLL הראשון. לאחר מכן, כל DLL נטען לכתובת גבוהה יותר בזיכרון. מרגע ש-DLL כלשהו נטען לכתובת בזיכרון, זו תהיה הכתובת במרחב הזיכרון עבור כל ה-Processים שישתמשו בו. אם נצפה בכתובת של DLL כלשהו ולאחר מכן נבצע אתחול למחשב שלנו, לאחר העליה הכתובת תשתנה.